

DSA + Git & GitHub + MERN + Machine Coding Round

Sheet 4 in the series — a fresh set of questions, none repeated from any earlier sheet. DSA questions are listed plainly with their LeetCode number — search and solve them directly on LeetCode. Git & GitHub and MERN questions are framed as **real interview scenarios** with follow-up questions that dig into your answer. The Machine Coding assignment is kept exactly as written — full problem statement, requirements, and deliverables, unchanged.

5 DSA

5 GIT

2 MERN

1 MACHINE CODING

HOW TO USE THIS SHEET

- DSA: search the LC number directly on leetcode.com/problems and solve it there.
- Git & MERN: read only the scenario first — pause, and try to answer out loud before reading further.
- Then check the follow-up questions — these are what a good interviewer asks next based on your answer.
- The Machine Coding section is a full build assignment, not a Q&A — treat it as a timed task.

SECTION 1 — DSA (LeetCode)

Solve these directly on LeetCode — search the question number, solve it there.

01 **Best Time to Buy and Sell Stock**

EASY

LC 121

02 **Maximum Subarray**

MEDIUM

LC 53

03 **Product of Array Except Self**

MEDIUM

LC 238

04 **Merge Intervals**

MEDIUM

LC 56

05 **Validate Binary Search Tree**

MEDIUM

LC 98

Search the LC number on leetcode.com/problems to find and solve each question directly.

SECTION 2 — Git & GitHub

Real situations from day-to-day dev work — not textbook definitions.

1. Fixing a Commit Message After the PR Is Already Open

MEDIUM

■ THE SCENARIO

You just pushed a commit with the message "asdfasdf" to your feature branch, and the PR is already open with two reviewers assigned.

■ INTERVIEWER ASKS

How do you fix the message without messing up the PR?

■ IF YOU ANSWER WELL, THEY DIG DEEPER

- What changes about the commit's hash once you amend it, and does that affect the open PR?
- What would you do differently if this commit wasn't the most recent one on the branch?
- How do you avoid confusing reviewers who already started reviewing before you amended?

2. A File Keeps Showing Up as Modified Even After Adding It to .gitignore

EASY

■ THE SCENARIO

A teammate added `config/local.json` to `.gitignore`, but it keeps showing up as modified in `git status` every time they edit it.

■ INTERVIEWER ASKS

What's actually going on, and how do you fix it for good?

■ IF YOU ANSWER WELL, THEY DIG DEEPER

- Why doesn't `.gitignore` affect a file that's already tracked?
- What's the exact command to stop tracking it without deleting it from disk?
- What would you check to make sure nobody re-adds it by accident later?

3. Recovering a Stash You Just Dropped

MEDIUM

■ THE SCENARIO

A teammate ran `git stash drop` and immediately realized they needed those changes back.

■ INTERVIEWER ASKS

Is that work actually gone, and if not, how do you get it back?

■ IF YOU ANSWER WELL, THEY DIG DEEPER

- Where does a dropped stash actually live before it's garbage collected?
- How would you identify the right stash commit among several dangling ones?
- What would you tell them to do differently next time to avoid this scare?

4. Recovering Commits After a Hard Reset Gone Wrong

HARD

■ THE SCENARIO

You meant to run `git reset --soft HEAD~1` to undo your last commit, but typed `--hard` instead, and now three hours of work seems to have vanished.

■ INTERVIEWER ASKS

Walk me through how you'd get that work back.

■ IF YOU ANSWER WELL, THEY DIG DEEPER

- Why does this work even though the commits aren't reachable from any branch anymore?
- What's the time window before this kind of recovery becomes impossible?
- How would you build a habit or safeguard so this mistake doesn't cost real time again?

5. Finding Who Introduced a Buggy Line

MEDIUM

■ THE SCENARIO

A specific line in a config file is causing a production issue, and nobody remembers when or why it was added.

■ INTERVIEWER ASKS

How would you trace exactly which commit introduced that line, and why?

■ IF YOU ANSWER WELL, THEY DIG DEEPER

- How is this different from just using bisect for the same problem?
- What do you do if the line has been touched by several commits since it was introduced?
- How would you present what you found to the team without pointing fingers?

SECTION 3 — MERN Stack Interview

Situational engineering problems — Express and Node.

Express.js

1. An API Getting Hammered by One Client

MEDIUM

■ THE SCENARIO

Your public API is getting flooded with requests from a single misbehaving script, slowing down responses for every other legitimate user.

■ INTERVIEWER ASKS

How would you protect the API without blocking real users?

■ IF YOU ANSWER WELL, THEY DIG DEEPER

- Where would you store request counts if your API runs on multiple server instances behind a load balancer?
- What would you actually return to a client that's been rate limited, and what status code?
- How would you rate limit more strictly on expensive endpoints, like search, than on cheap ones?

Node.js

2. The Server Slows Down More Every Day

HARD

■ THE SCENARIO

A Node.js API has been running in production for three weeks without a restart, and average response times have crept up steadily the entire time — a restart fixes it instantly, but only temporarily.

■ INTERVIEWER ASKS

How would you actually diagnose what's happening?

■ IF YOU ANSWER WELL, THEY DIG DEEPER

- What tools would you reach for to confirm this is a memory leak and not something else?
- What are the most common causes of a leak like this in an Express app specifically?
- How would you narrow down which route or feature is responsible without guessing?

SECTION 4 — Machine Coding Round

Not a Q&A — a full build assignment, kept exactly as originally written.

1. MetricLens — Team Analytics Dashboard

MEDIUM

3 Hrs

Build an analytics dashboard for a SaaS product where team admins can see usage metrics — active users, feature adoption, and engagement trends — pulled from raw event data.

- Auth: register, login, logout, protected routes, admin vs. member roles
- Ingest and store raw usage events (userId, eventType, timestamp, metadata) in MongoDB
- Aggregation-powered dashboard: daily active users, top features, retention over time
- Date-range filtering and per-team scoping, with no cross-team data leakage
- Chart-ready API endpoints (time series + breakdowns) consumed by the React frontend
- Backend validation + edge cases (empty date ranges, teams with zero events)

React.js

Node.js

Express.js

MongoDB